Linux File Permissions and ACLs

Read, write and execute for owner, group and others - plus Access Control Lists for fine-grained control. With examples, command chart and interview Q&A.

ROOT = changes the system or reads protected data; run it from a root shell. **OWN FILES** = changes only files you own; run it as a normal user in your own folders. **ANY USER** = read-only, safe for everyone.

How to read this: examples are written without sudo, as your class set them. That does not make them all harmless - the labels above tell you which are safe. Replace words in <angle brackets> with real names.

1. What file permissions are

Every file and directory in Linux carries a set of permissions that decides who may open it, change it, or run it. This is the first layer of security and access control on the system: before any program touches a file, the kernel checks these bits.

There are three permission **types**:

Command	What it does	Example
`r - read`	View the contents. On a directory: list the names inside it (ls).	`cat file.txt`
`w - write`	Modify the file's contents. On a directory: create, rename or delete the entries inside it.	`edit file.txt`
`x - execute`	Run the file as a program or script. On a directory: enter it (cd) and reach its files.	`./script.sh`

And three **categories of users** each type can be granted to:

Owner (u) - the user who owns the file. **Group** (g) - every user in the file's group. **Others** (o) - everyone else on the system.

One point classes often get wrong: **write permission on a file does not let you delete it**. Deleting (or renaming) a file changes the directory that holds it, so what you need is **write and execute permission on the containing directory** - the file's own permissions are not even checked for deletion. The exception is a directory with the **sticky bit** set, like /tmp: there everyone can create files, but you can delete only files you own yourself - the directory's owner and root are not restricted. Set it with chmod +t.

2. Reading the permission string (ls -l)

List files with a long listing to see permissions:

```
# ANY USER - view current permissions of a file
ls -l filename
-rw-r--r-- 1 sumanth students 2048 Nov  5 10:00 notes.txt
```

The first column, `-rw-r--r--`, is ten characters. Break it down like this:

The **first character** is the file type: `-` for a regular file, `d` for a directory, `l` for a symbolic link. The next three characters are the **owner's** rwx, the middle three the **group's**, and the last three **others'**. A dash in place of a letter means that permission is off.

So `-rw-r--r--` means: a regular file; the owner may read and write; the group may only read; others may only read.

3. Changing permissions with chmod

The `chmod` command changes file permissions, and there are two ways to say what you want: symbolic mode and numeric mode.

Symbolic mode

Use letters for the category (u, g, o, or a for all), an operator to **add** (+), **remove** (-) or **set exactly** (=), and the permission letters:

```
# OWN FILES - changes only your own files
chmod g+w file.txt     # add write permission for the group
chmod o-r file.txt     # remove read permission from others
chmod u+x script.sh    # make a script runnable by its owner
chmod a=r file.txt     # everyone gets exactly read, nothing else
```

Numeric mode

Each permission is a number: read = 4, write = 2, execute = 1. Add them per category to get one digit, then write three digits: owner, group, others.

```
# OWN FILES
chmod 754 file.txt     # owner 7 = rwx, group 5 = r-x, others 4 = r--
```

Command	What it does	Example
`chmod 644`	rw- r-- r-- : owner edits, everyone reads. Typical for notes and web files.	`chmod 644 a.txt`
`chmod 755`	rwx r-x r-x : owner edits and runs, everyone can run. Typical for scripts and directories.	`chmod 755 run.sh`
`chmod 700`	rwx --- --- : owner only. Private directories.	`chmod 700 ~/private`
`chmod 600`	rw- --- --- : owner reads/writes only. Private files.	`chmod 600 secret.txt`

4. Changing ownership (chown and chgrp)

Permissions attach to an owner and a group, so two related commands change who those are. `chown` changes the owner (and optionally the group); `chgrp` changes only the group.

```
# ROOT - only root can hand a file to another owner
chown ravi file.txt            # make ravi the owner
chown ravi:students file.txt   # owner ravi, group students
# OWN FILES - change the group to one you are a member of
chgrp students file.txt
```

5. Access Control Lists (ACLs)

Traditional permissions stop at one owner, one group, and everyone else. Real setups often need more: for example, Asha and Ravi both need write access to a file, but they are in different groups. **Access Control Lists** answer this - they let you set permissions for multiple named users and groups on the same file or directory, which is especially useful in collaborative environments.

An ACL is a list of entries. Each entry names a specific user or group and grants read, write and/or execute, using the same rwx letters as standard permissions. Entries cover: specific **users**, specific **groups**, and an **other** entry for anyone not listed. A file with an ACL shows a + at the end of its permission string in ls -l.

Viewing ACLs

```
# ANY USER - show the ACL of a file or directory
getfacl myfile.txt
```

Modifying ACLs

```
# OWN FILES
# give user bob read and write on myfile.txt
setfacl -m u:bob:rw myfile.txt
# give group auditors read access
setfacl -m g:auditors:r myfile.txt
```

The mask and effective rights

An ACL carries one more entry: the **mask**. The mask is a ceiling - it caps what named users, named groups and the owning group can actually get. When the mask clips an entry, getfacl shows the truth beside it, for example `user:bob:rwx  #effective:r-x`: bob was granted rwx but effectively has only r-x.

Two practical rules: running chmod on the **group** bits of a file that has an ACL changes the **mask**, not the group entry - this is the usual reason ACL entries mysteriously stop working. And if the mask got tightened by accident, set it back with `setfacl -m m::rwx myfile.txt`.

Removing entries

```
# remove one entry
setfacl -x u:bob myfile.txt
# remove the whole ACL, back to plain permissions
setfacl -b myfile.txt
```

Command chart

Command	What it does	Example
`ls -l file`	Show the permission string, owner and group	`ls -l notes.txt`
`chmod g+w f`	Symbolic: add write for the group (OWN FILES)	`chmod o-r f`
`chmod 754 f`	Numeric: rwx r-x r-- (OWN FILES)	`chmod 644 f`
`chown u f`	Change file owner (ROOT)	`chown ravi f`
`chgrp g f`	Change file group (OWN FILES, to a group you are in)	`chgrp students f`
`getfacl f`	Show the ACL entries	`getfacl myfile.txt`
`setfacl -m`	Add or change an ACL entry (OWN FILES)	`setfacl -m u:bob:rw f`
`setfacl -x / -b`	Remove one entry / the whole ACL (OWN FILES)	`setfacl -x u:bob f`

Interview Q&A

Q. What do the three permission types mean?

Read lets you view a file (or list a directory), write lets you change its contents (or change a directory's entries), and execute lets you run it as a program (or enter a directory).

Q. How do you read -rw-r--r-- ?

First character is the file type (- = regular file). Then three groups of three: owner has read+write, group has read only, others have read only.

Q. What does chmod 754 do?

Owner gets 7 = 4+2+1 = rwx, group gets 5 = 4+1 = r-x, others get 4 = r--.

Q. Symbolic vs numeric chmod - when would you use each?

Symbolic is precise for small changes (chmod g+w adds one bit without touching the rest). Numeric sets all nine bits at once, so it is better when you want an exact final state.

Q. Why use ACLs when permissions already exist?

Standard permissions allow only one owner and one group. ACLs let you grant rights to several named users and groups on the same file - for example giving bob rw while the group keeps only r.

Q. Can you delete a file you have write permission on?

Not from the file’s permissions alone. Deletion is decided by write plus execute on the containing directory. On a sticky-bit directory like /tmp you can delete only files you own; the directory owner and root are not restricted by the sticky bit.

Q. What is the ACL mask?

A ceiling on the rights of named users, named groups and the owning group. getfacl shows #effective: rights when the mask is tighter than the entry. chmod on the group bits moves the mask on a file that has an ACL.

Q. How do you see and change a file’s ACL?

getfacl shows it. setfacl -m u:username:perms adds or changes an entry, setfacl -x removes one entry, setfacl -b removes the entire ACL.

Sources: class slides (File Permission, ACL Permission) and the Linux man pages (man chmod, man chown, man getfacl, man setfacl).